

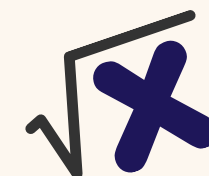
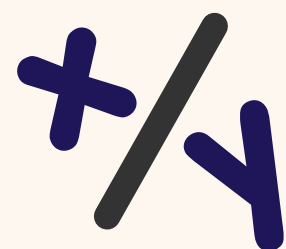
$f(x)$



PROYECTO #2

6 API - CALCULADORA 5

Proyecto realizado por Edwin Moreno y Sara Lee



ÍNDICE

- 01. Diagrama de Arquitectura de Alto Nivel
- 02. Aspectos Generales del Código Fuente
- 03. API, Código Fuente y Repositorio

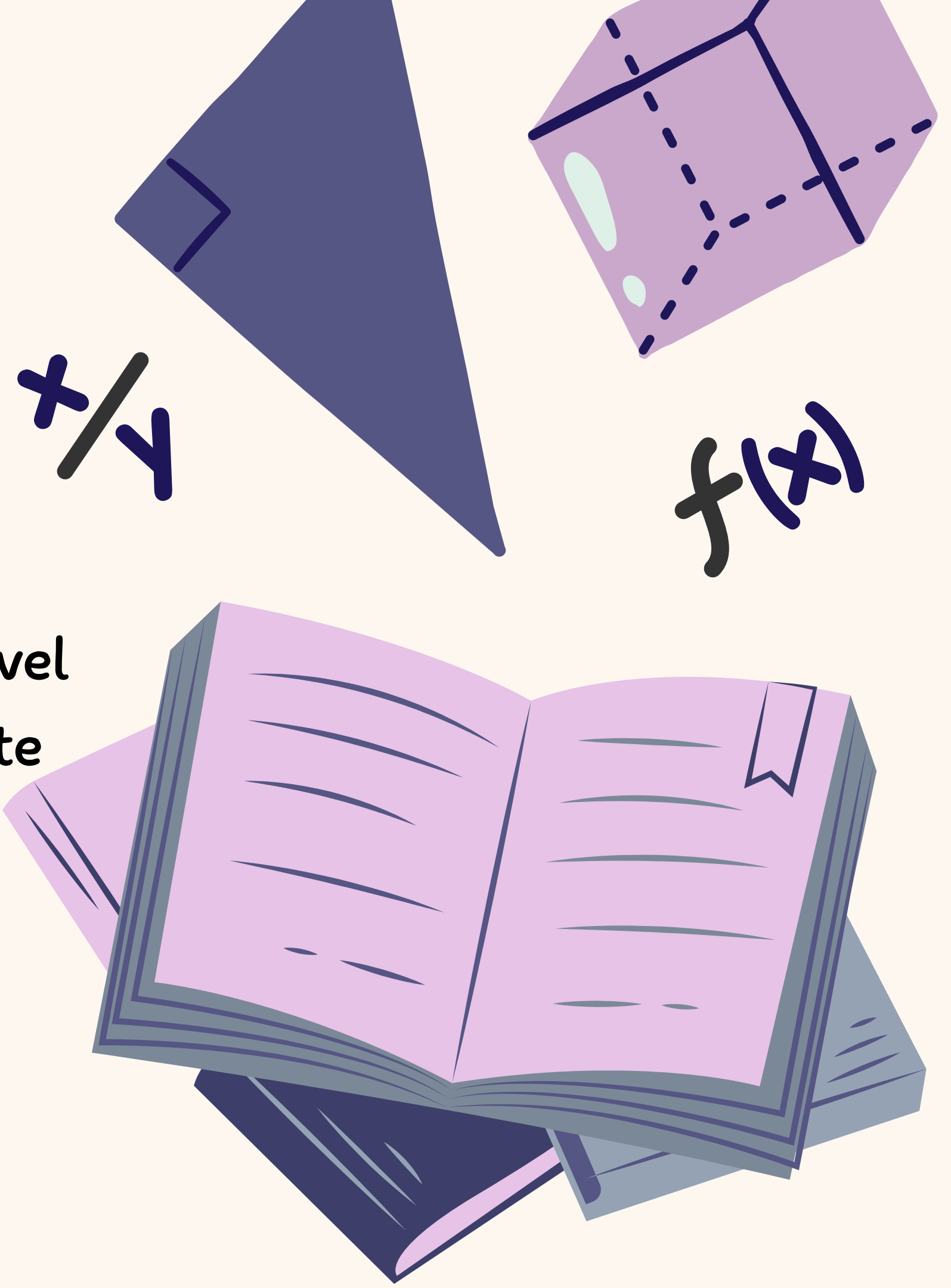
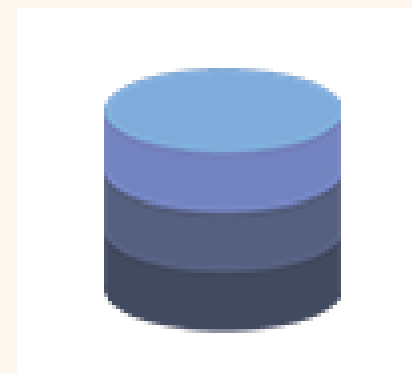
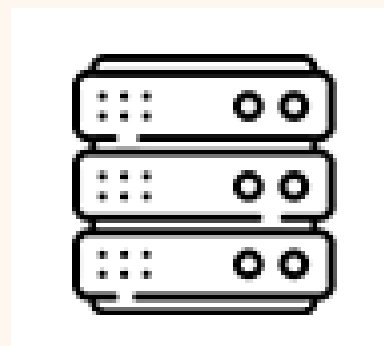
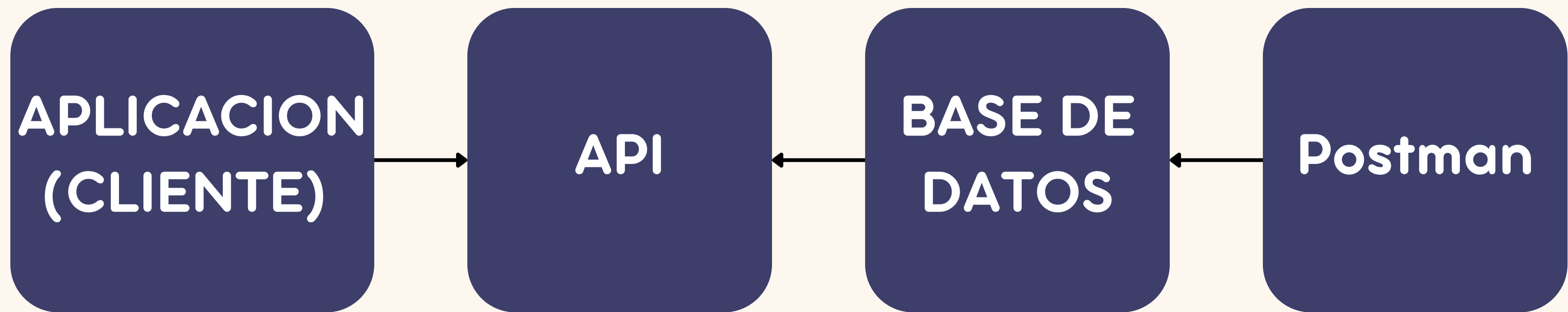
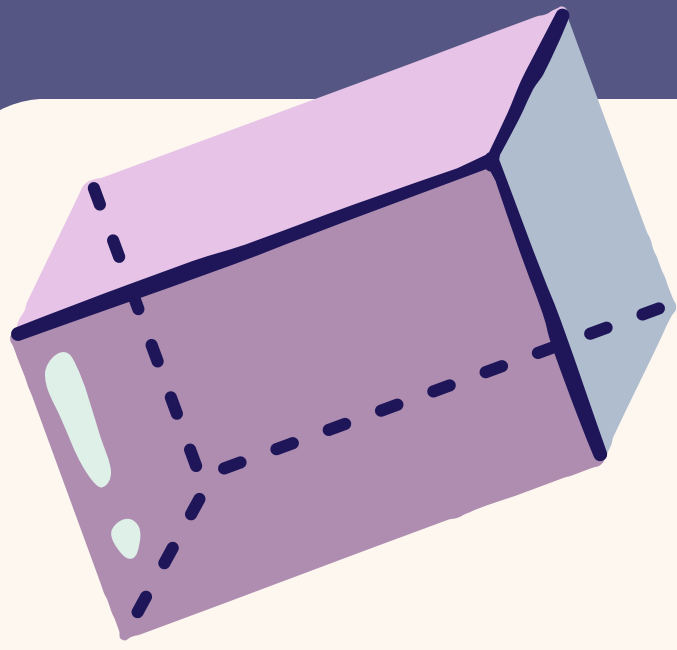
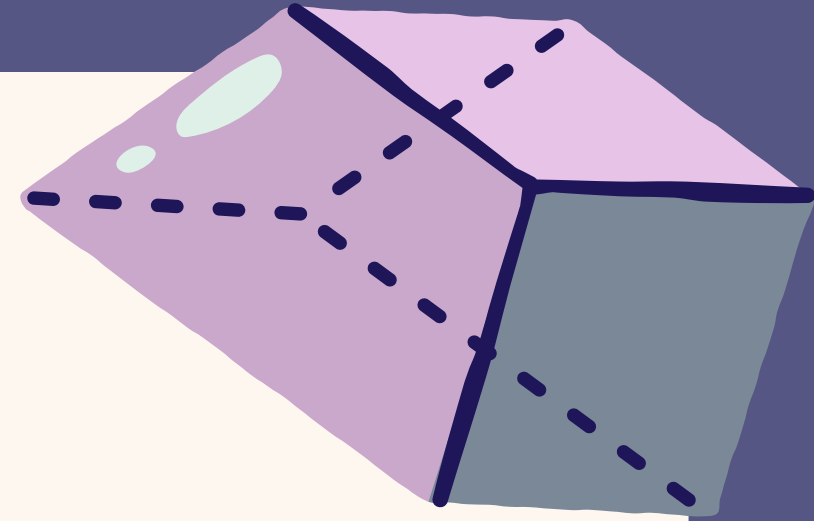


DIAGRAMA DE ARQUITECTURA



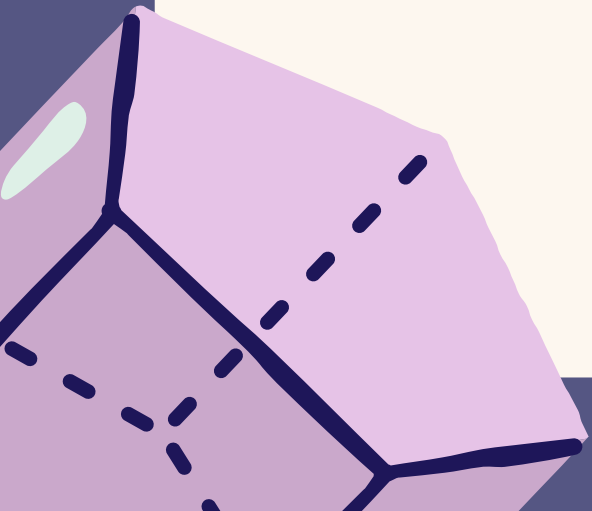


$$\sqrt{x}$$

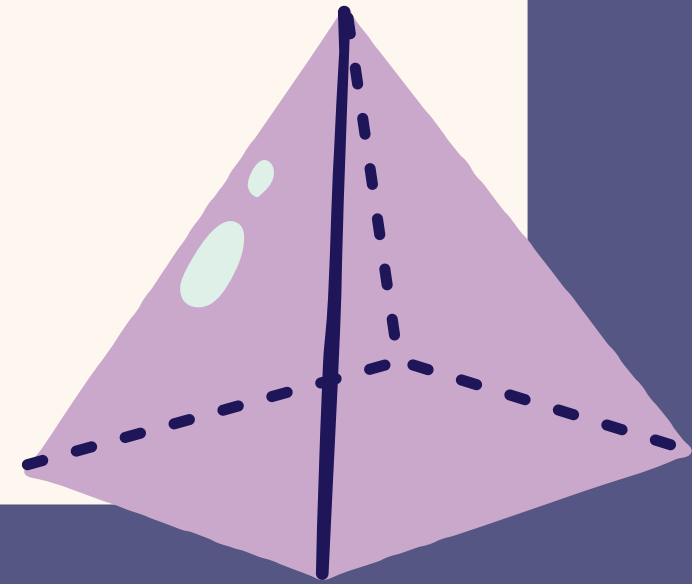


$$\frac{x}{y}$$

ASPECTOS GENERALES DEL CODIGO FUENTE



$$f(x)$$



► C# CalculosController.cs

```
51 [HttpGet]
52 [Route("Sumas")]
53 0 references
54 public IActionResult GetSumas()
55 {
56     List<Calculo> sumas = new List<Calculo>();
57     try
58     {
59         using (SqlConnection conn = DBConnection.GetConnection())
60         {
61             conn.Open();
62             string query = "SELECT * FROM Calculos WHERE Operacion = '+' ORDER BY FechaCalc";
63             using (SqlCommand cmd = new SqlCommand(query, conn))
64             {
65                 using (SqlDataReader reader = cmd.ExecuteReader())
66                 {
67                     while (reader.Read())
68                     {
69                         sumas.Add(new Calculo
70                         {
71                             Id = Convert.ToInt32(reader["Id"]),
72                             Numero1 = Convert.ToDouble(reader["Numero1"]),
73                             Numero2 = Convert.ToDouble(reader["Numero2"]),
74                             Operacion = reader["Operacion"].ToString(),
75                             Resultado = Convert.ToDouble(reader["Resultado"]),
76                             FechaCalculo = Convert.ToDateTime(reader["FechaCalculo"])
77                         });
78                     }
79                 }
80             }
81             return Ok(sumas);
82         }
83     } catch (Exception ex)
```

Métodos GET
filtran operaciones
por tipo: sumas,
restas,
multiplicaciones,
divisiones,
potencias.

► C# CalculosController.cs

```
[HttpPost]
0 references
public IActionResult PostCalculo(Calculo calculo)
{
    try
    {
        // Calcular el resultado según la operación
        switch (calculo.Operacion)
        {
            case "+":
                calculo.Resultado = calculo.Numero1 + calculo.Numero2;
                break;
            case "-":
                calculo.Resultado = calculo.Numero1 - calculo.Numero2;
                break;
            case "*":
                calculo.Resultado = calculo.Numero1 * calculo.Numero2;
                break;
            case "/":
                if (calculo.Numero2 == 0)
                    return BadRequest("No se puede dividir por cero");
                calculo.Resultado = calculo.Numero1 / calculo.Numero2;
                break;
            case "^":
                calculo.Resultado = Math.Pow(calculo.Numero1, calculo.Numero2);
                break;
            default:
                return BadRequest("Operación no válida");
        }
    }
}
```

Método POST permite insertar un cálculo en la base de datos.

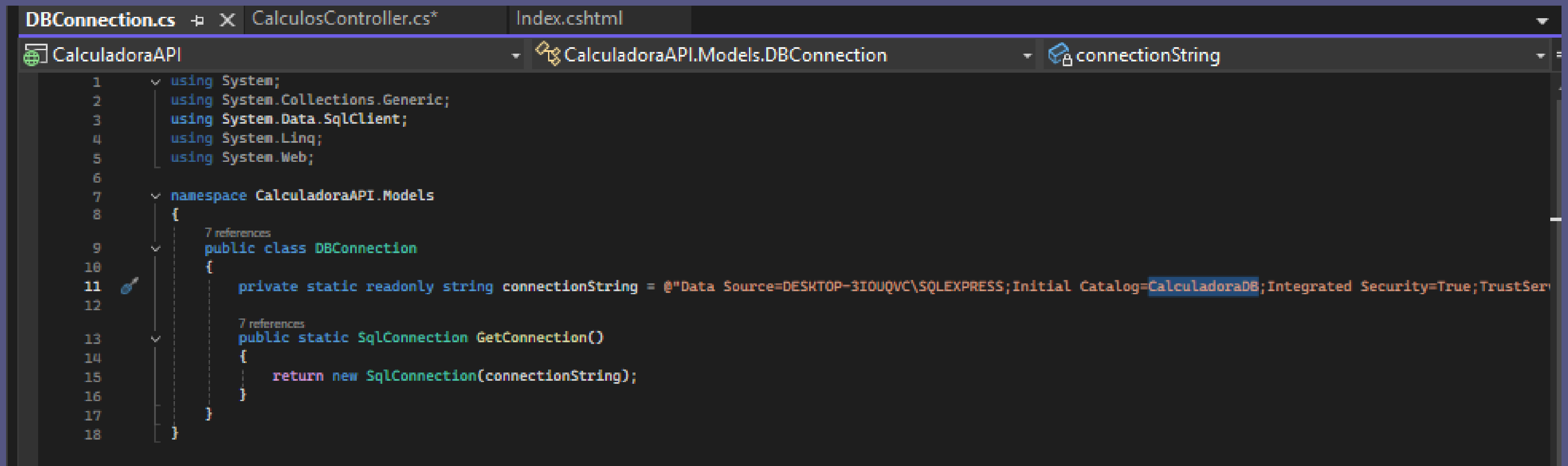
```
using (SqlConnection conn = DBConnection.GetConnection())
{
    conn.Open();
    string query = @"INSERT INTO Calculos (Numero1, Numero2, Operacion, Resultado)
                    VALUES (@Numero1, @Numero2, @Operacion, @Resultado)";

    using (SqlCommand cmd = new SqlCommand(query, conn))
    {
        cmd.Parameters.AddWithValue("@Numero1", calculo.Numero1);
        cmd.Parameters.AddWithValue("@Numero2", calculo.Numero2);
        cmd.Parameters.AddWithValue("@Operacion", calculo.Operacion);
        cmd.Parameters.AddWithValue("@Resultado", calculo.Resultado);
        cmd.ExecuteNonQuery();
    }

    return Ok(calculo);
}
catch (Exception ex)
{
    return InternalServerError(ex);
}
```

► A C# DBConnection.cs

Conexion a la base de datos, usamos la misma que en el proyecto #1



```
DBConnection.cs  X CalculosController.cs*  Index.cshtml
CalculadoraAPI  CalculadoraAPI.Models.DBConnection  connectionString
1  using System;
2  using System.Collections.Generic;
3  using System.Data.SqlClient;
4  using System.Linq;
5  using System.Web;
6
7  namespace CalculadoraAPI.Models
8  {
9      7 references
10     public class DBConnection
11     {
12         private static readonly string connectionString = @"Data Source=DESKTOP-3IOUQVC\SQLEXPRESS;Initial Catalog=CalculadoraDB;Integrated Security=True;TrustSer
13
14         7 references
15         public static SqlConnection GetConnection()
16         {
17             return new SqlConnection(connectionString);
18         }
19     }
20 }
```



DBConnection.cs | Git Repositor...dwin02 (main) | CalculosController.cs | Index.cshtml

edwin02 (main) | Filter | Branches / Tags

edwin02 (main) .. ↑↓ 0 / 0
remotes/ori...

Branch / Tag: main

g Graph Message Author Date ID

g (0) Fetch | Pull

History (0 Outgoing) Push | Sync

	Corrigien...	sara lee	11/27/202...	7e080e5d
	Eliminaci...	sara lee	11/26/202...	b56d51d9
	Correccio...	sara lee	11/26/202...	30d4d1a3
	Correccio...	sara lee	11/26/202...	7576129b
	reemplaz...	sara lee	11/26/202...	7f31fc22
	.	sara lee	11/26/202...	0b35baa5
	Restauran...	edwinnC	11/26/202...	917b8d43
	Añadir do...	edwinnC	11/24/202...	e237675e
	Añadir lab...	edwinnC	11/22/202...	e8d3515f
	Añadir lab...	edwinnC	11/17/202...	ae6477c5
	Merge pul...	edwinnC	11/12/202...	39fefe39
	proyecto...	edwinnC	11/12/202...	bff46c16

¡GRACIAS!